

BIG DATA ANALYTICS

UNIT – II : Introduction to Hadoop

Introducing Hadoop, need of Hadoop, limitations of RDBMS, RDBMS versus Hadoop, Distributed Computing Challenges, History of Hadoop , Hadoop Overview, Use Case of Hadoop, Hadoop Distributors, HDFS, Processing Data with Hadoop, Managing Resources and Applications with Hadoop YARN, Interacting with Hadoop Ecosystem.

Explain the differences between Hadoop and RDBMS

Parameters	RDBMS	Hadoop
System	Relational Database Management system	Node based flat structure
Data	Suitable for structured data	Suitable for Structured, unstructured data, supports variety of formats(xml,json)
Processing	OLTP	Analytical, big data processing
Choice	When the data needs consistent relationship	Big data processing, which does not require any consistent relationships between data
Processor	Needs expensive hardware or high-end processors to store huge volumes of data	In a Hadoop clusters, node require any consistent relationships between data
Cost	Cost around \$10,000 to \$14,000 per terabytes of storage	Cost around \$4000 per terabytes of storage.

What is Hadoop? Explain features of hadoop.

Hadoop is an open source framework that is meant for storage and processing of big data in a distributed manner.

It is the best solution for handling big data challenges.

Some important features of Hadoop are –

Open Source – Hadoop is an open source framework which means it is available free of cost. Also, the users are allowed to change the source code as per their requirements.

Distributed Processing – Hadoop supports distributed processing of data i.e. faster processing. The data in Hadoop HDFS is stored in a distributed manner and MapReduce is responsible for the parallel processing of data.

Fault Tolerance – Hadoop is highly fault-tolerant. It creates three replicas for each block (default) at different nodes.

Reliability – Hadoop stores data on the cluster in a reliable manner that is independent of machine. So, the data stored in Hadoop environment is not affected by the failure of the machine.

Scalability – It is compatible with the other hardware and we can easily add the new hardware to the nodes.

High Availability – The data stored in Hadoop is available to access even after the hardware failure. In case of hardware failure, the data can be accessed from another node.

The core components of Hadoop are –

HDFS: (Hadoop Distributed File System) – HDFS is the basic storage system of Hadoop. The large data files running on a cluster of commodity hardware are stored in HDFS. It can store data in a reliable manner even when hardware fails.

The key aspects of HDFS are:

Storage component

Distributes data across several nodes

Natively redundant.

Map Reduce: MapReduce is the Hadoop layer that is responsible for data processing. It writes an application to process unstructured and structured data stored in HDFS.

It is responsible for the parallel processing of high volume of data by dividing data into independent tasks. The processing is done in two phases Map and Reduce.

The Map is the first phase of processing that specifies complex logic code and the

Reduce is the second phase of processing that specifies light-weight operations.

The key aspects of Map Reduce are:

Computational frame work

Splits a task across multiple nodes

Processes data in parallel

Explain features of HDFS. Discuss the design of Hadoop distributed file system and concept in detail.

HDFS: (Hadoop Distributed File System) – HDFS is the basic storage system of Hadoop. The large data files running on a cluster of commodity hardware are stored in HDFS. It can store data in a reliable manner even when hardware fails.

The key aspects of HDFS are:

HDFS is developed by the inspiration of Google File System(GFS).

Storage component: Stores data in hadoop

Distributes data across several nodes: divides large file into blocks and stores in various data nodes.

Natively redundant: replicates the blocks in various data nodes.

High Throughput Access: Provides access to data blocks which are nearer to the client.

Re-replicates the nodes when nodes are failed.

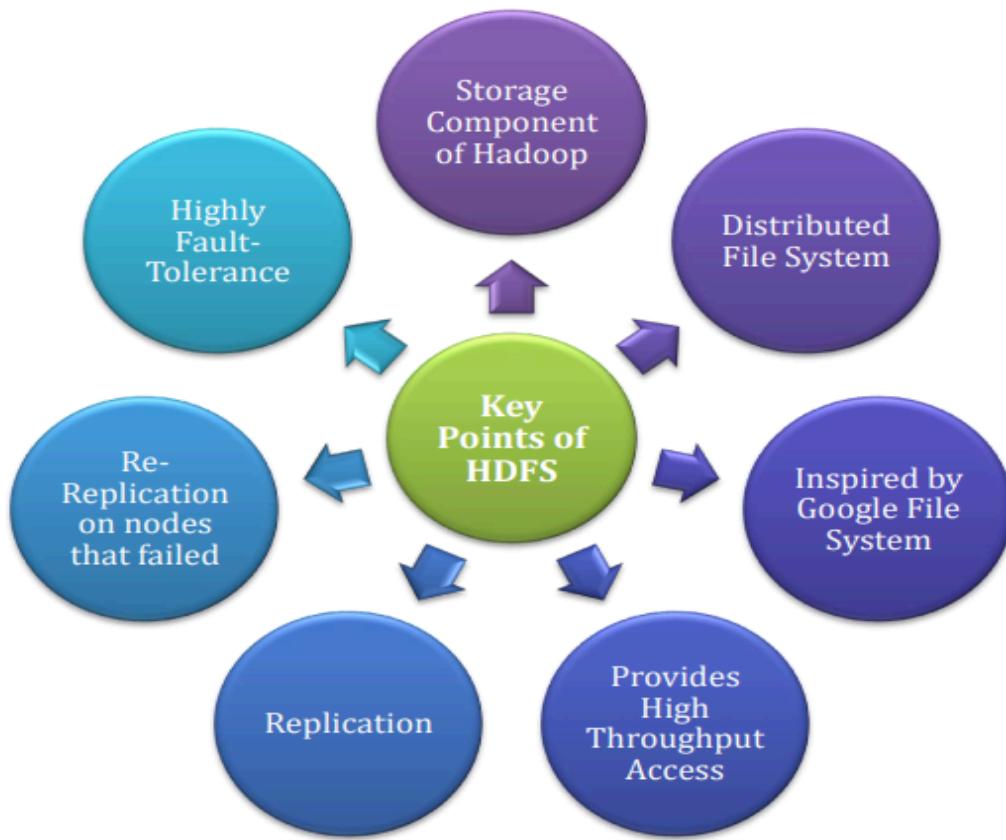


Fig. Features of HDFS

HDFS Daemons:

NameNode

The NameNode is the master of HDFS that directs the slave DataNodes to perform I/O tasks.

Blocks: HDFS breaks large file into smaller pieces called blocks.

rackID: NameNode uses rackID to identify data nodes in the rack.

(rack is a collection of datanodes with in the cluster) NameNode keep track of blocks of a file.

File System Namespace: NameNode is the book keeper of HDFS. It keeps track of how files are broken down into blocks and which DataNode stores these blocks. It is a collection of files in the cluster.

FsImage: file system namespace includes mapping of blocks of a file, file properties and is stored in a file called FsImage.

EditLog: namenode uses an EditLog (transaction log) to record every transaction that happens to the file system metadata.

NameNode is single point of failure of Hadoop cluster.

HDFS KEY POINTS		
BLOCK STRUCTURED FILE	DEFAULT REPLICATION FACTOR: 3	DEFAULT BLOCK SIZE: 64MB/128MB

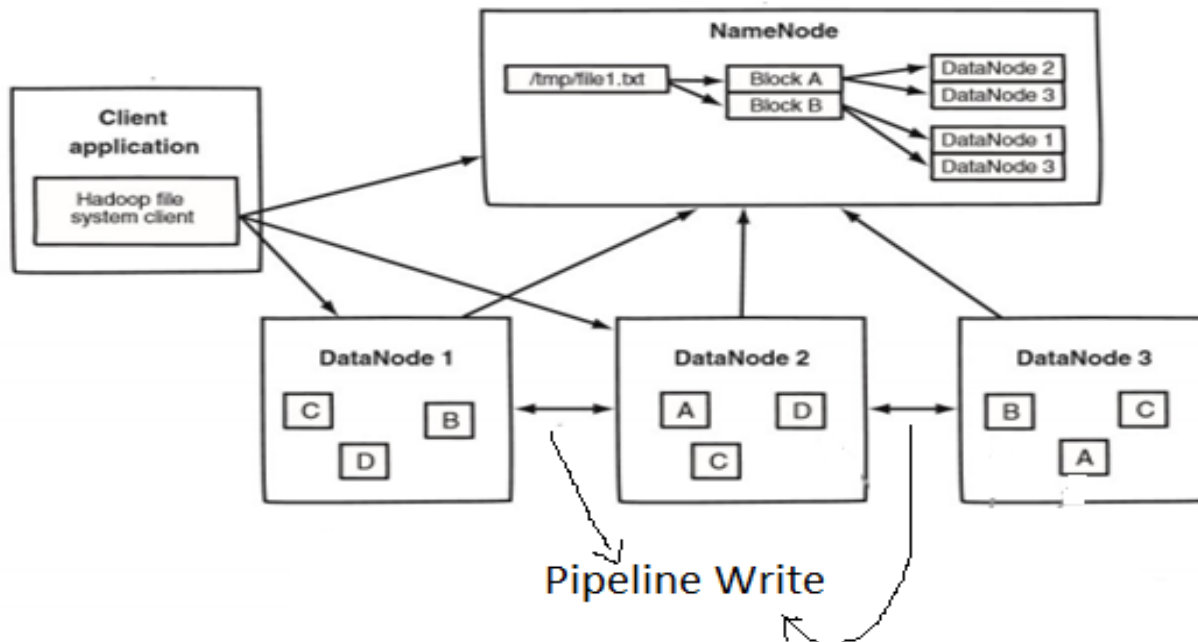


Fig. HDFS Architecture

DataNode

Multiple data nodes per cluster. Each slave machine in the cluster have DataNode daemon for reading and writing HDFS blocks of actual file on local file system

During pipeline read and write DataNodes communicate with each other.

It also continuously Sends "heartbeat" message to NameNode to ensure the connectivity between the Name node and the data node.

If no heartbeat is received for a period of time NameNode assumes that the DataNode had failed and it is re-replicated.

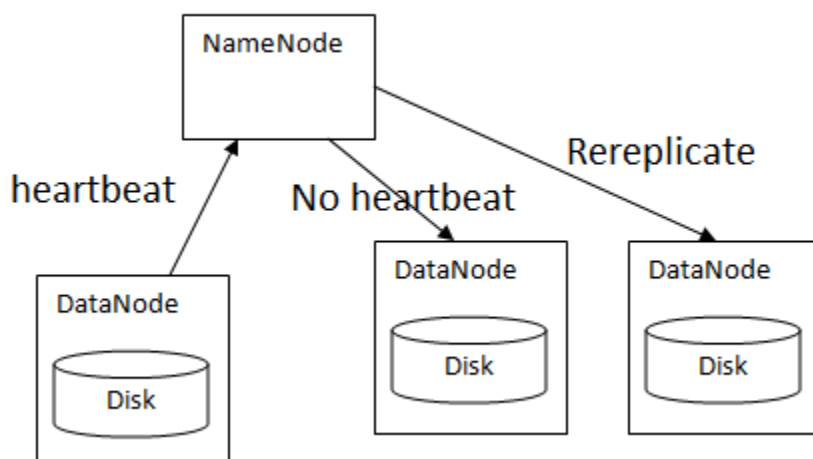


Fig. Interaction between NameNode and DataNode.

(iii) Secondary name node

Takes snapshot of HDFS meta data at intervals specified in the hadoop configuration.

Memory is same for secondary node as NameNode.

But secondary node will run on a different machine.

In case of name node failure secondary name node can be configured manually to bring up the cluster but we cannot make secondary namenode as name node.

Read and Write operations in Hadoop (VVVI):

File Read operation:

The steps involved in the File Read are as follows:

The client opens the file that it wishes to read from by calling `open()` on the DFS.

The DFS communicates with the NameNode to get the location of data blocks. NameNode returns with the addresses of the DataNodes that the data blocks are stored on.

Subsequent to this, the DFS returns an FSD to client to read from the file.

Client then calls `read()` on the stream `DFSInputStream`, which has addresses of DataNodes for the first few block of the file.

Client calls `read()` repeatedly to stream the data from the DataNode.

When the end of the block is reached, `DFSInputStream` closes the connection with the DataNode. It repeats the steps to find the best DataNode for the next block and subsequent blocks.

When the client completes the reading of the file, it calls `close()` on the `FSInputStream` to the connection.

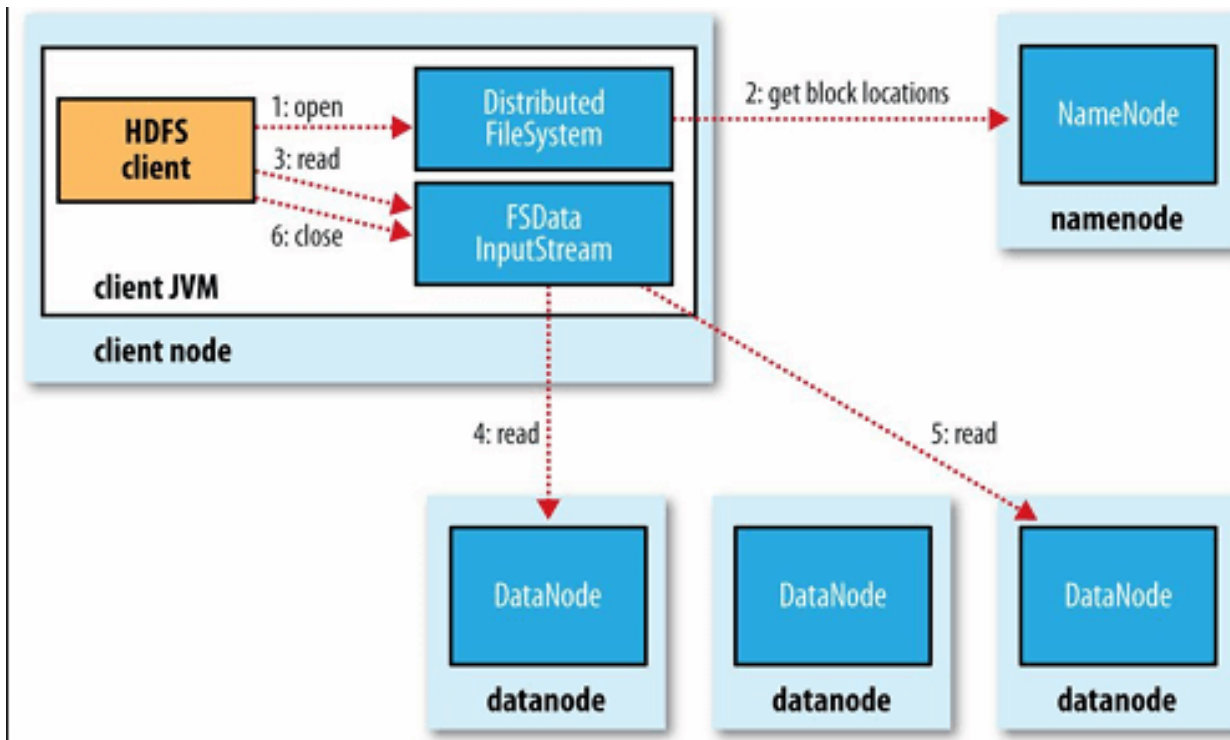


Fig. File Read Anatomy

File Write operation:

The client calls create() on DistributedFileSystem to create a file.

An RPC call to the namenode happens through the DFS to create a new file.

As the client writes data, data is split into packets by DFSOutputStream, which is then writes to an internal queue, called data queue. Datastreamer consumes the data queue.

Data streamer streams the packets to the first DataNode in the pipeline. It stores packet and forwards it to the second DataNode in the pipeline.

In addition to the internal queue, DFSOutputStream also manages on "Ackqueue" of the packets that are waiting for acknowledged by DataNodes.

When the client finishes writing the file, it calls close() on the stream.

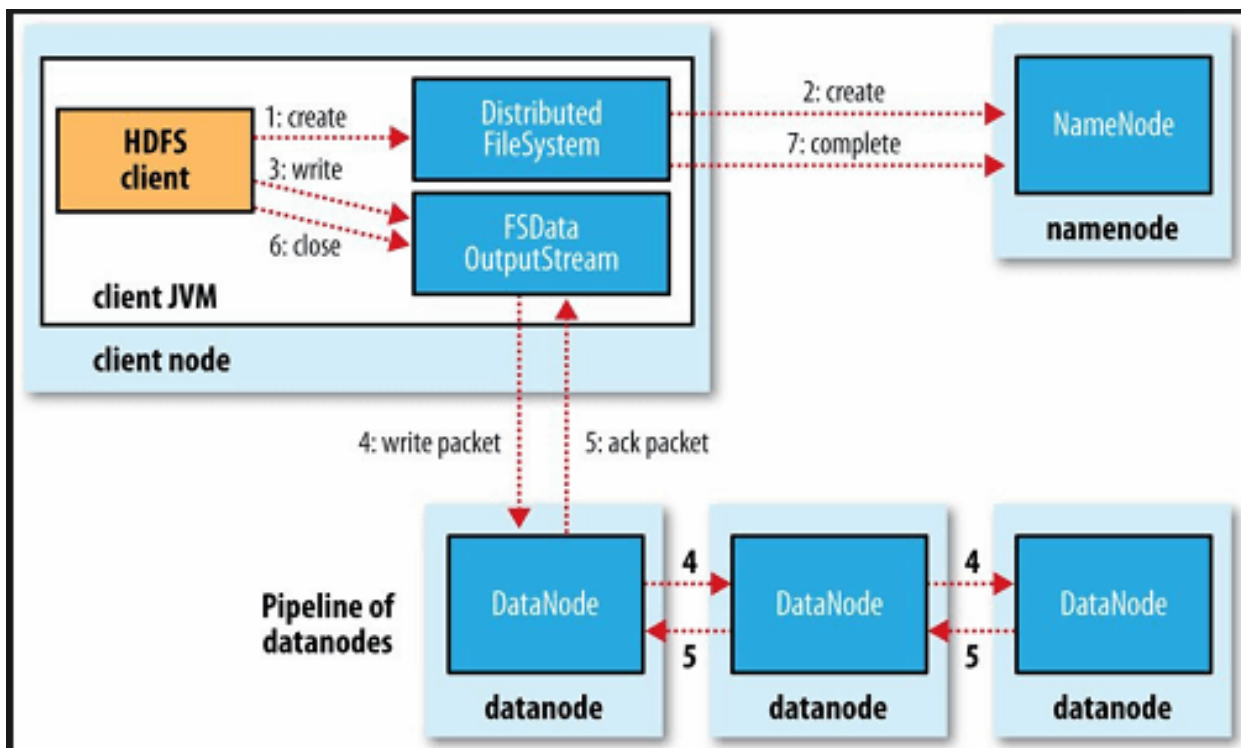


Fig. File Write Anatomy

Special features of HDFS:

Data Replication: There is absolutely no need for a client application to track all blocks. It directs client to the nearest replica to ensure high performance.

Data Pipeline: A client application writes a block to the first DataNode in the pipeline. Then this DataNode takes over and forwards the data to the next node in the pipeline. This process continues for all the data blocks, and subsequently all the replicas are written to the disk.

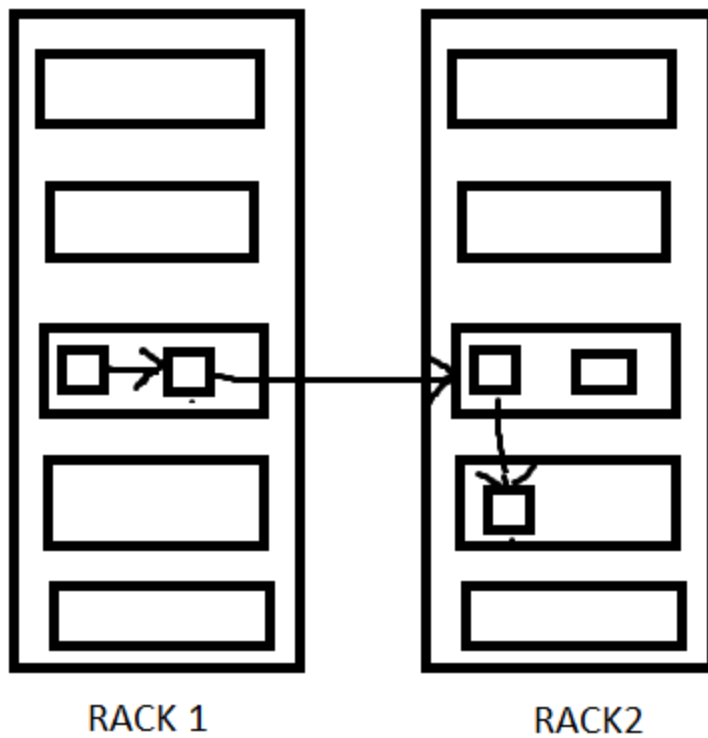


Fig. File Replacement Strategy

Explain basic HDFS File operations with an example (VVVVI).

Creating a directory:

Syntax: `hdfs dfs -mkdir <path>`

Eg. `hdfs dfs -mkdir /chp`

Remove a file in specified path:

Syntax: `hdfs dfs -rm <src>`

Eg. `hdfs dfs -rm /chp/abc.txt`

Copy file from local file system to hdfs:

Syntax: `hdfs dfs -copyFromLocal <src> <dst>`

Eg. `hdfs dfs dfs -copyFromLocal /home/vignan/sample.txt /chp/abc1.txt`

To display list of contents in a directory:

Syntax: `hdfs dfs -ls <path>`

Eg. `hdfs dfs -ls /chp`

To display contents in a file:

Syntax: `hdfs dfs -cat <path>`

Eg. `hdfs dfs -cat /chp/abc1.txt`

Copy file from hdfs to local file system:

Syntax: `hdfs dfs -copyToLocal <src <dst>`

Eg. `hdfs dfs -copyToLocal /chp/abc1.txt /home/vignan/Desktop/sample.txt`

To display last few lines of a file:

Syntax: `hdfs dfs -tail <path>`

Eg. `hdfs dfs -tail /chp/abc1.txt`

Display aggregate length of file in bytes:

Syntax: `hdfs dfs -du <path>`

Eg. `hdfs dfs -du /chp`

To count no.of directories, files and bytes under given path:

Syntax: `hdfs dfs -count <path>`

Eg. `hdfs dfs -count /chp`

o/p: 1 1 60

Remove a directory from hdfs

Syntax: `hdfs dfs -rmr <path>`

Eg. `hdfs dfs rmr /chp`

Explain difference between Hadoop1X and Hadoop2X

Limitations of Hadoop 1.0: HDFS and MapReduce are core components, while other components are built around the core.

Single namenode is responsible for entire namespace.

It is Restricted processing model which is suitable for batch-oriented mapreduce jobs.

Not supported for interactive analysis.

Not suitable for Machine learning algorithms, graphs, and other memory intensive algorithms

MapReduce is responsible for cluster resource management and data Processing.

HDFS Limitation: The NameNode can quickly become overwhelmed with load on the system increasing. In Hadoop 2.x this problem is resolved.

Hadoop 2: Hadoop 2.x is YARN based architecture. It is general processing platform. YARN is not constrained to MapReduce only. One can run multiple applications in Hadoop 2.x in which all applications share common resource management.

Hadoop 2.x can be used for various types of processing such as Batch, Interactive, Online, Streaming, Graph and others.

DFS 2 consists of two major components

a) NameSpace: Takes care of file related operations such as creating files, modifying files and directories

b) Block storage service: It handles data node cluster management and replication.

HDFS 2 Features:

Horizontal scalability: HDFS Federation uses multiple independent NameNodes for horizontal scalability. The DataNodes are common storage for blocks and shared by all NameNodes. All DataNodes in the cluster registers with each NameNode in the cluster.

High availability: High availability of NameNode is obtained with the help of Passive Standby NameNode.

Active-Passive NameNode handles failover automatically. All namespace edits are recorded to a shared NFS(Network File Storage) Storage and there is a single writer at a point of time.

Passive NameNode reads edits from shared storage and keeps updated metadata information.

In case of Active NameNode failure, Passive NameNode becomes an Active NameNode automatically. Then it starts writing to the shared storage.

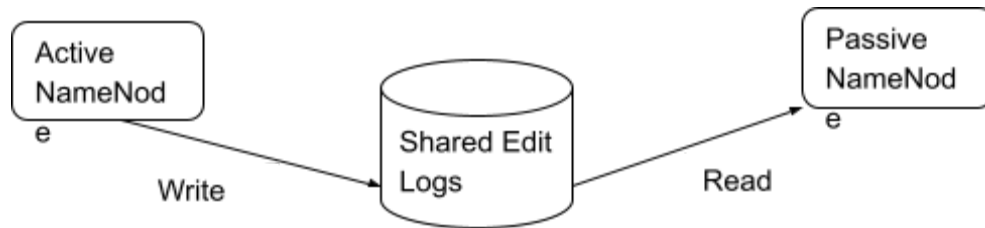
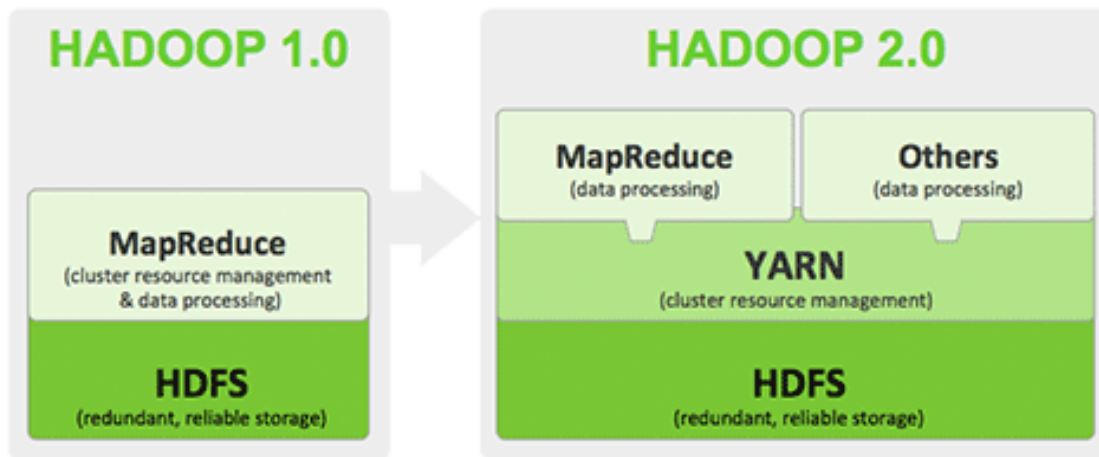


Fig. Active and Passive NameNode Interaction



	Hadoop1X	Hadoop2X
1	Supports MapReduce (MR) processing model only. Does not support non-MR tools	Allows to work in MR as well as other distributed computing models like Spark, & HBase coprocessors.
2	MR does both processing and cluster-resource management.	YARN does cluster resource management and processing is done using different processing models.
3	Has limited scaling of nodes. Limited to 4000 nodes per cluster	Has better scalability. Scalable up to 10000 nodes per cluster
4	Works on concepts of slots – slots can run either a Map task or a Reduce task only.	Works on concepts of containers. Using containers can run generic tasks.
5	A single Namenode to manage the entire namespace.	Multiple Namenode servers manage multiple namespaces.
6	Has Single-Point-of-Failure (SPOF) – because of single Namenode.	Has to feature to overcome SPOF with a standby Namenode and in the case of Namenode failure, it is configured for automatic recovery.
7	MR API is compatible with Hadoop1x. A program written in Hadoop1 executes in Hadoop1x without any additional files.	MR API requires additional files for a program written in Hadoop1x to execute in Hadoop2x.
8	Has a limitation to serve as a platform for event processing, streaming and real-time operations.	Can serve as a platform for a wide variety of data analytics-possible to run event processing, streaming and real-time operations.
9	Does not support Microsoft Windows	Added support for Microsoft windows

Explain in detail about YARN (VVVI: short notes)?

The fundamental idea behind the YARN(Yet Another Resource Negotiator) architecture is to splitting the JobTracker responsibility of resource management and job scheduling/monitoring into separate daemons.

Daemons that are part of YARN architecture are:

1. Global Resource Manager: The main responsibility of Global Resource Manager is to distribute resources among various applications.

It has two main components:

Scheduler: The pluggable scheduler of ResourceManager decides allocation of resources to various running applications. The scheduler is just that, a pure scheduler, meaning it does NOT monitor or track the status of the application.

Application Manager: It does:

Accepting job submissions.

Negotiating resources(container) for executing the application specific ApplicationMaster

Restarting the ApplicationMaster in case of failure

NodeManager:

This is a per-machine slave daemon. NodeManager responsibility is launching the application containers for application execution.

NodeManager monitors the resource usage such as memory, CPU, disk, network, etc.

It then reports the usage of resources to the global ResourceManager.

Per-Application Application Master: Per-application Application master is an application specific entity. It's responsibility is to negotiate required resources for execution from the ResourceManager.

It works along with the NodeManager for executing and monitoring component tasks.

Basic concepts of YARN are: Application and Container.

Application is a job submitted to system.

Ex: MapReduce job.

Container: Basic unit of allocation. Replaces fixed map/reduce slots. Fine-grained resource allocation across multiple resource type

Eg. Container_0: 2GB,1CPU

Container_1: 1GB,6CPU

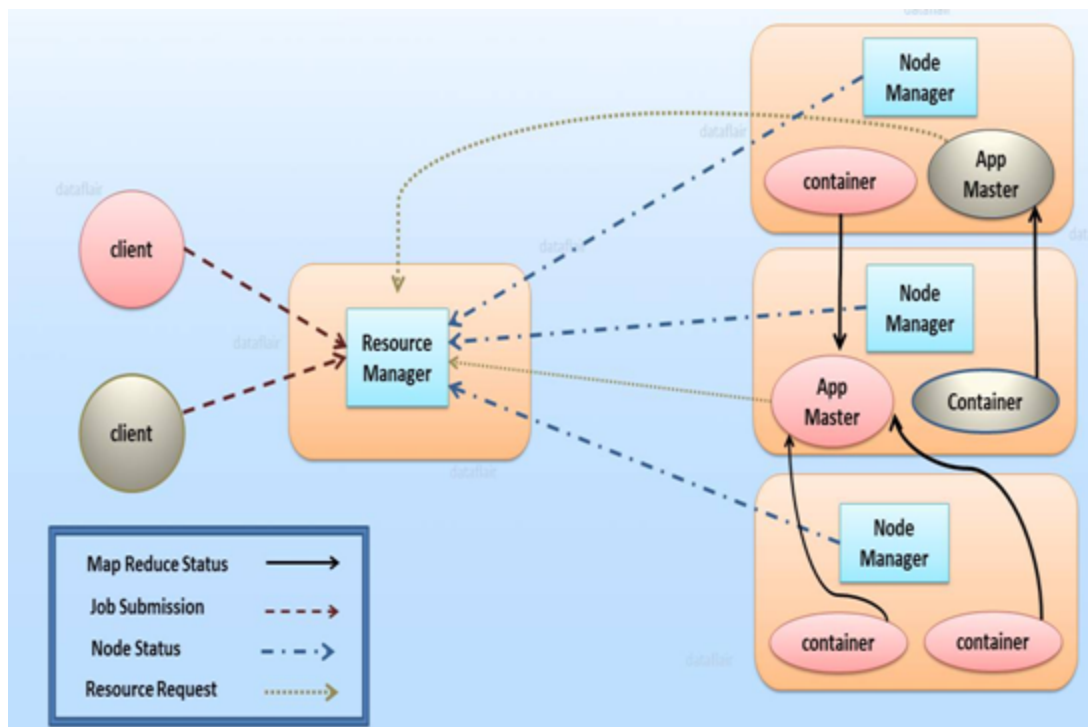


Fig. YARN Architecture

The steps involved in YARN architecture are:

The client program submits an application.

The Resource Manager launches the Application Master by assigning some container.

The Application Master registers with the Resource manager.

On successful container allocations, the application master launches the container by providing the container launch specification to the NodeManager.

The NodeManager executes the application code.

During the application execution, the client that submitted the job directly communicates with the Application Master to get status, progress updates.

Once the application has been processed completely, the application master deregisters with the ResourceManager and shutdown allowing its own container to be repurposed.

Explain Hadoop Ecosystem in detail.

The following are the components of Hadoop ecosystem:

HDFS: Hadoop Distributed File System. It simply stores data files as close to the original form as possible.

HBase: It is Hadoop's distributed column based database. It supports structured data storage for large tables.

Hive: It is a Hadoop's data warehouse, enables analysis of large data sets using a language very similar to SQL. So, one can access data stored in hadoop cluster by using Hive.

Pig: Pig is an easy to understand data flow language. It helps with the analysis of large data sets which is quite the order with Hadoop without writing codes in MapReduce paradigm.

ZooKeeper: It is an open source application that configures synchronizes the distributed systems.

Oozie: It is a workflowscheduler system to manage apache hadoop jobs.

Mahout: It is a scalable Machine Learning and data mining library.

Chukwa: It is a data collection system for managing large distributed systems.

Sqoop: it is used to transfer bulk data between Hadoop and structured data stores such as relational databases.

Ambari: it is a web based tool for provisioning, Managing and Monitoring Apache Hadoop clusters.

Explain the following

Modules of Apache Hadoop framework

There are four basic or core components:

Hadoop Common: It is a set of common utilities and libraries which handle other Hadoop modules. It makes sure that the hardware failures are managed by Hadoop cluster automatically.

Hadoop YARN: It allocates resources which in turn allow different users to execute various applications without worrying about the increased workloads.

HDFS: It is a Hadoop Distributed File System that stores data in the form of small memory blocks and distributes them across the cluster. Each data is replicated multiple times to ensure data availability.

Hadoop MapReduce: It executes tasks in a parallel fashion by distributing the data as small blocks.

Hadoop Modes of Installations/configurations

Standalone, or local mode: which is one of the least commonly used environments, which only for running and debugging of MapReduce programs. This mode does not use HDFS nor it launches any of the hadoop daemon.

Pseudo-distributed mode(Cluster of One), which runs all daemons on single machine. It is most commonly used in development environments.

Fully distributed mode, which is most commonly used in production environments. This mode runs all daemons on a cluster of machines rather than single one.

XML File configurations in Hadoop.

core-site.xml – This configuration file contains Hadoop core configuration settings, for example, I/O settings, very common for MapReduce and HDFS. mapred-site.xml – This configuration file specifies a framework name for MapReduce by setting mapreduce.framework.name

hdfs-site.xml – This configuration file contains HDFS daemons configuration settings. It also specifies default block permission and replication checking on HDFS.

yarn-site.xml – This configuration file specifies configuration settings for ResourceManager and NodeManager.

Describe differences between SQL and MapReduce

Characteristic	SQL	MapReduce(Hadoop1X)
Access	Interactive and Batch	Batch
Structure	Static	Dynamic
Updates	Read and Write many times	Write once, Read many times
Integrity	High	Low
Scalability	Nonlinear	Linear

Explain Hadoop Architecture with a neat sketch.

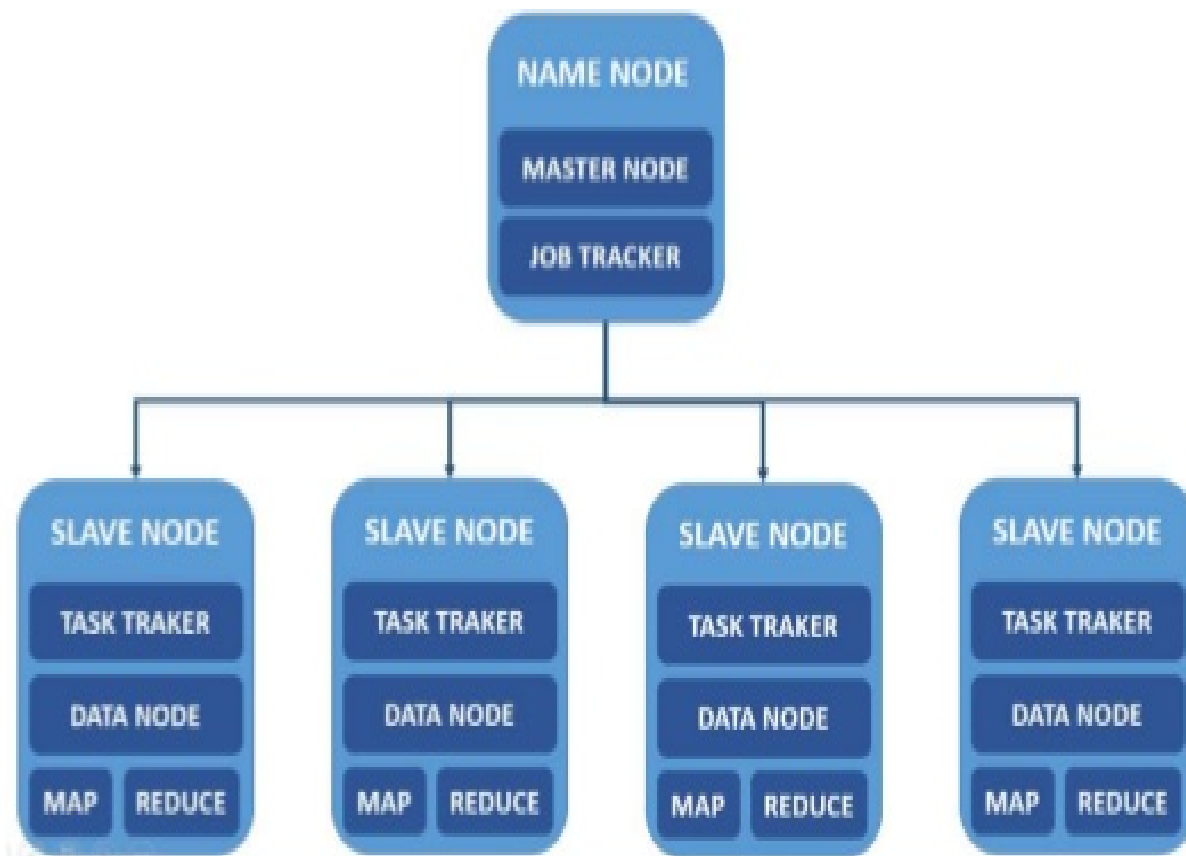


Fig. Hadoop Architecture

Hadoop Architecture is a distributed Master-slave architecture.

Master HDFS: Its main responsibility is partitioning the data storage across the slave nodes. It also keep track of locations of data on Datanodes.

Master Map Reduce: It decides and schedules computation task on slave nodes.

NOTE: Based on marks for the question explain hdfs daemons and mapreduce daemons.